

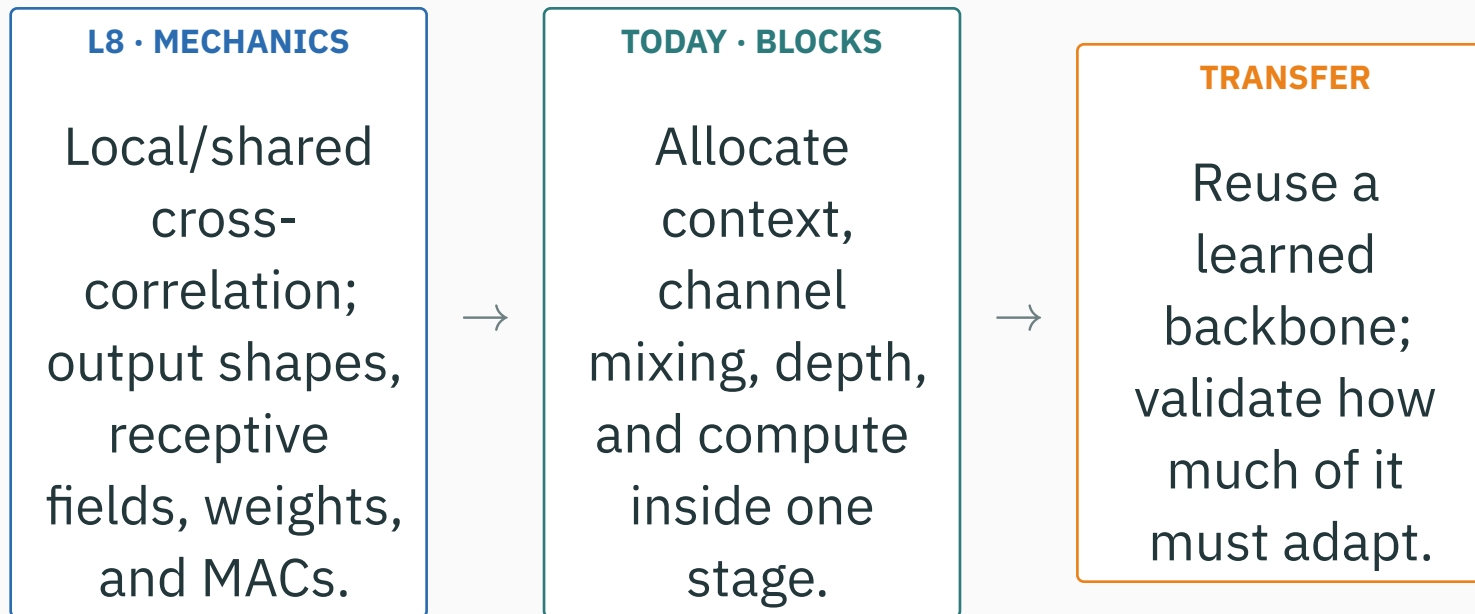
Modern CNN Pipelines and Transfer Learning

Blocks, Scaling, Residuals, and Reusing Backbones

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

L8 gave us the operator; today we spend it deliberately



L6 already established why residual routes help optimization. L7 fixed the validation/test protocol. We will retrieve both—then use them inside a deployable CNN pipeline.

Commit: which design can meet the stage budget?

QUESTION

HELD CASE · DEPLOYMENT CONTRACT $28 \times 28 \times 64 \rightarrow 28 \times 28 \times 128$, same spatial grid, receptive field at least 3×3 , fewer than 10 million convolution MACs.

A · STANDARD

one shared 3×3 , $64 \rightarrow 128$

B · FACTORIZED

separate spatial filtering
from channel mixing

C · EITHER

sharing alone guarantees
the budget

Commit to A, B, or C. Write the count you would calculate first; keep the choice until the closing revisit.

The student contract and 80-minute route

By the end, you should be able to turn a block diagram into a shape/compute ledger, justify one design move, and run a leak-free probe-first transfer experiment.

CORE · 55 MIN

bridge → context +
bottleneck → residuals →
separable convolution →
transfer → revisit

SHOULD · 15 MIN

Inception branches →
frozen state +
preprocessing →
diagnostic tests

OPTIONAL · 10 MIN

groups + scaling +
ConvNeXt → FPN teaser
→ primary provenance

— held input — learned operation — context / shape — compute / adaptation — verified — failure

One ledger will carry every design decision

HELD CASE · LEDGER CONVENTION All stage candidates use $H_{\text{out}} = W_{\text{out}} = 28$, stride 1, stated same padding, and no bias. One MAC means one multiply-accumulate per weight per output location.

candidate	output	RF	weights	MACs
standard 3×3	$28 \times 28 \times$ 128	3×3	?	?

Budget: fewer than 10,000,000 MACs. Shape alone is not enough.

Every candidate must preserve the output contract; only context, factorization, route, and cost may change.

Baseline: count the standard 3×3 before optimizing

QUESTION

HELD CASE · CANDIDATE 0 One same-padded 3×3 convolution maps $64 \rightarrow 128$ channels at 28×28 output locations. Bias is omitted.

$$\text{weights} = 3 \cdot 3 \cdot 64 \cdot 128$$

$$\text{MACs} = H_{\text{out}} W_{\text{out}} \cdot \text{weights}$$

Predict both exact counts and whether the design meets the 10M budget.

Baseline answer: sharing is necessary but not sufficient

A ANSWER

candidate	output	RF	weights	MACs
standard 3×3	$28 \times 28 \times 128$	3×3	73, 728	57, 802, 752

$57.803\text{M} > 10\text{M}$: the output contract passes; the deployment budget fails.

WHAT L8 BOUGHT

One kernel bank is reused at all 784 locations.

WHAT REMAINS

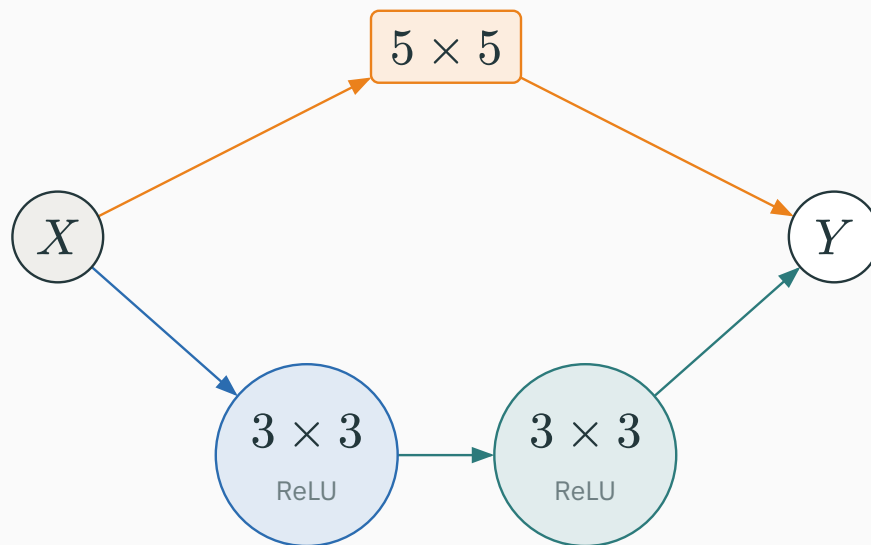
Reduce work inside the kernel bank without changing the output.

The lecture is now a controlled search for a cheaper stage—not a tour of architecture names.

Design move 1 — grow context with small kernels

HELD CASE · CONTEXT Keep the held $28 \times 28 \times 64 \rightarrow 28 \times 28 \times 128$ output. Now require a 5×5 receptive field.

Should we use one 5×5 convolution—or compose two same-padded 3×3 s?



Both paths see the same spatial extent. Only the blue–teal path inserts an extra learned nonlinearity.

Receptive field: derive it one layer at a time

With stride 1, each $k \times k$ layer grows the receptive-field side length by $k - 1$:

$$r_\ell = r_{\ell-1} + (k - 1)$$
$$1 \xrightarrow{3 \times 3} 3 \xrightarrow{3 \times 3} 5 \xrightarrow{3 \times 3} 7 \xrightarrow{3 \times 3} 9$$

Four stacked 3×3 layers match one 9×9 receptive field.

They also insert **four** transformations and **four** activations instead of one.

Depth grows context gradually—and learns what to keep after each step.

Same output and context: count both candidates

The stacked path uses $64 \rightarrow 64 \rightarrow 128$; the direct path uses $64 \rightarrow 128$. Biases are omitted.

	two 3×3 s	one 5×5
weights	$9(64 \cdot 64 + 64 \cdot 128)$	$25 \cdot 64 \cdot 128$
count	110, 592	204, 800
ReLUs	2	1

$204,800 - 110,592 = 94,208$ fewer weights: a 46% reduction.

Small kernels are not merely aesthetic: they buy equal context, fewer weights, and more nonlinear stages.

Ledger: more context still misses the deployment budget

D DERIVATION

candidate	output	RF	weights	MACs
standard 3×3	$28 \times 28 \times 128$	3×3	73,728	57.803M
two 3×3 s	$28 \times 28 \times 128$	5×5	110,592	86.704M
one 5×5	$28 \times 28 \times 128$	5×5	204,800	160.563M

All three preserve shape with stated same padding. None meets 10M.

Stacking small kernels is a context decision. It does not, by itself, solve the channel-pair cost.

Misconception check: does depth always grow spatial context?

QUESTION

Stack ten 1×1 convolutions. What is the receptive field?

A · 1×1

B · 10×10

C · depends on channels

Answer: A. Depth composes transformations; kernel size and stride determine spatial reach.

Design move 2 — mix channels without mixing neighbours

At one spatial location, collect the C_{in} channel values into a vector x_{hw} :

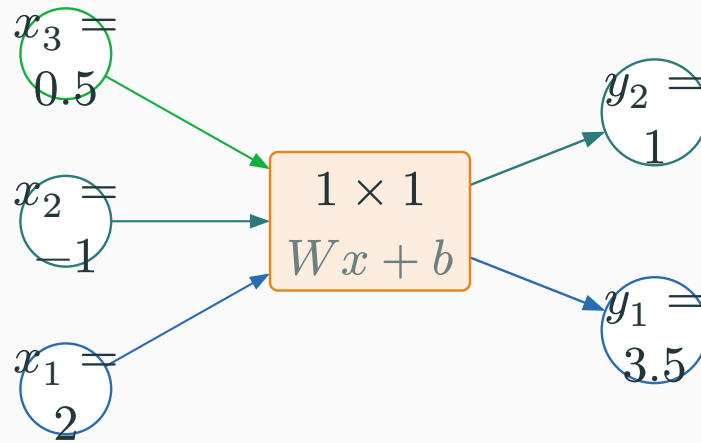
$$y_{hw} = Wx_{hw} + b, \quad W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$$

The **same** matrix W is used at every location.

No value from $(h + 1, w)$ or $(h, w + 1)$ is read.

A 1×1 convolution is a shared, per-pixel linear layer across channels.

One pixel, two learned channel combinations



For $x = (2, -1, 0.5)^\top$, choose $W = \begin{pmatrix} 1 & -1 & 0 \\ 0.5 & 0 & 2 \end{pmatrix}$ and $b = (0.5, -1)^\top$.

$$y = \begin{pmatrix} 1 & -1 & 0 \\ 0.5 & 0 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0.5 \\ -1 \end{pmatrix} = \begin{pmatrix} 3.5 \\ 1 \end{pmatrix}.$$

Two output channels = two different learned summaries of the same input channels.

The same channel mixer runs over the whole grid

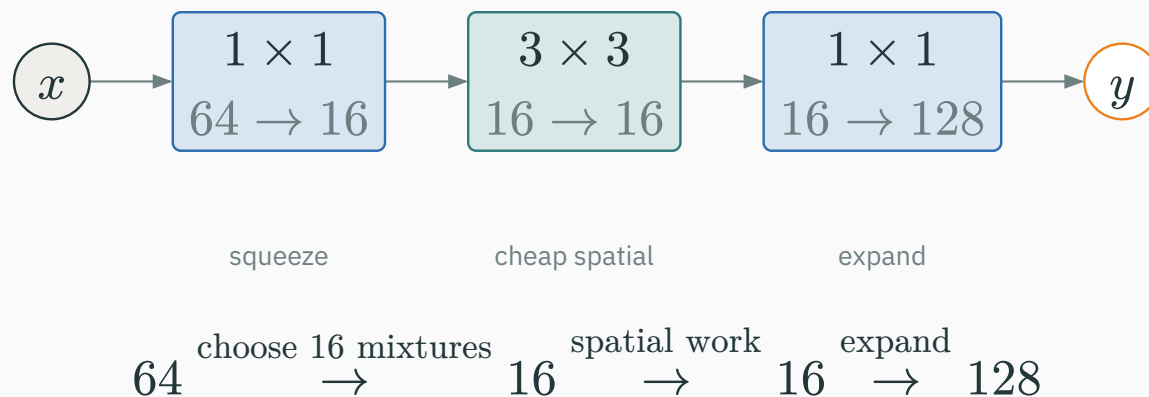
x_{hw}	$(0, 0)$	$(0, 1)$	$(1, 0)$	$(1, 1)$
three channels	$\begin{pmatrix} 2 \\ -1 \\ 0.5 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 1 \\ 2 \end{pmatrix}$	$\begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 3 \\ 0 \\ 1 \end{pmatrix}$
$Wx + b$	$\begin{pmatrix} 3.5 \\ 1 \end{pmatrix}$	$\begin{pmatrix} -0.5 \\ 3 \end{pmatrix}$	$\begin{pmatrix} -1.5 \\ -1.5 \end{pmatrix}$	$\begin{pmatrix} 3.5 \\ 2.5 \end{pmatrix}$

$$2 \times 2 \times 3 \xrightarrow{\text{same } 1 \times 1 \text{ weights}} 2 \times 2 \times 2$$

Spatial coordinates stay aligned; only the channel representation changes.

Bottleneck: spend the 3×3 in a narrow channel space

HELD CASE · CANDIDATE 2 Narrow $64 \rightarrow 16$, perform the spatial work at width 16, then expand $16 \rightarrow 128$. Same 28×28 grid throughout.



The two 1×1 s learn which channel combinations deserve expensive spatial processing.

For the held 28×28 stage, compare the direct $3 \times 3, 64 \rightarrow 128$, with $64 \rightarrow 16 \rightarrow 16 \rightarrow 128$:

direct weights $= 3 \cdot 3 \cdot 64 \cdot 128 = 73,728$.

With the bottleneck:

$64 \cdot 16 + 3 \cdot 3 \cdot 16 \cdot 16 + 16 \cdot 128 = 1,024 + 2,304 + 2,048 = 5,376$.

Each weight is reused at $28^2 = 784$ locations: 57.803 million versus 4.215 million MACs.

$5,376 \cdot 784 = 4,214,784$: the first candidate under the 10M budget.

Misconception check: what did the 1×1 save?

A ANSWER

It did **not** make the final 3×3 kernel spatially smaller.

wrong: $3 \times 3 \rightarrow 1 \times 1$ spatial window

right: $64 \rightarrow 16$ channels **before** the 3×3

The saving comes from fewer input–output channel pairs inside the expensive spatial convolution.

Ledger: channel compression changes the decision

candidate	output	RF	weights	MACs
standard 3×3	$28 \times 28 \times 128$	3×3	73,728	57.803M
two 3×3 s	$28 \times 28 \times 128$	5×5	110,592	86.704M
bottleneck 16	$28 \times 28 \times 128$	3×3	5,376	4.215M

GREEN means the candidate satisfies shape, receptive-field, and compute contracts.

The bottleneck clears the budget with 5.785M MACs to spare.

SHOULD · let one block inspect several scales

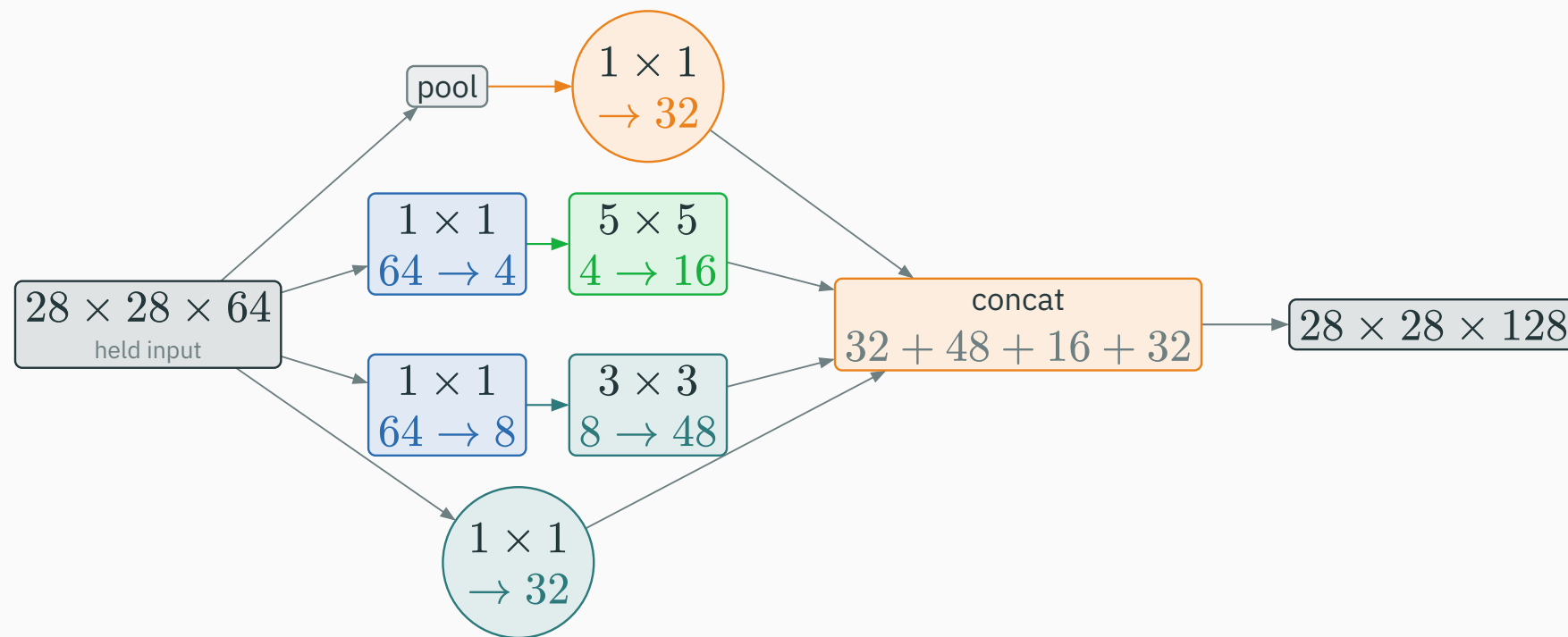
HELD CASE · CANDIDATE 3 The held stage may need channel evidence, 3×3 texture, 5×5 context, and a pooled summary at the same location.

Can four branches preserve the 28×28 grid, concatenate to 128 channels, and stay under budget?

1×1 channel evidence 3×3 local pattern 5×5 wider context pooling summary

Inception runs alternatives in parallel; the stage output keeps every branch as a channel group.

A shaped Inception candidate: every width is visible



$32 + 48 + 16 + 32 = 128$ output channels; height and width remain 28×28 .

Concatenation stacks channels; it does not average or add branch outputs.

Count the multi-scale candidate branch by branch

All convolutions use stride 1. The 3×3 , 5×5 , and 3×3 pool use padding 1, 2, and 1, so every branch remains 28×28 .

branch	weight expression	weights
$1 \times 1 \rightarrow 32$	$64 \cdot 32$	2,048
$1 \times 1 \rightarrow$ $8 \xrightarrow{3 \times 3} 48$	$64 \cdot 8 + 9 \cdot 8 \cdot 48$	3,968
$1 \times 1 \rightarrow$ $4 \xrightarrow{5 \times 5} 16$	$64 \cdot 4 + 25 \cdot 4 \cdot 16$	1,856
pool $\xrightarrow{1 \times 1} 32$	$64 \cdot 32$	2,048

9,920 weights and $9,920 \cdot 784 = 7,777,280$ convolution MACs. Pooling comparisons are excluded from this ledger.

Concatenate only after every spatial shape agrees

D DERIVATION

$28 \times 28 \times 32; \parallel; 28 \times 28 \times 48; \parallel; 28 \times 28 \times 16; \parallel; 28 \times 28 \times 32$

concatenate $\rightarrow 28 \times 28 \times (32 + 48 + 16 + 32) = 28 \times 28 \times 128$

The candidate meets shape and compute contracts; its widest branch sees a 5×5 neighbourhood.

Which statement is true?

A. Every signal crosses all four branches in sequence.

B. Each branch offers an alternative transform at one stage; concatenation preserves all outputs.

Answer: B. Branching widens the choice of scale; stacking modules supplies depth.

L6 retrieval · learn a correction, not a replacement

Suppose a useful block should mostly preserve its input. A plain block must learn $H(x) \approx x$.

A residual block instead writes

$$H(x) = x + F(x)$$

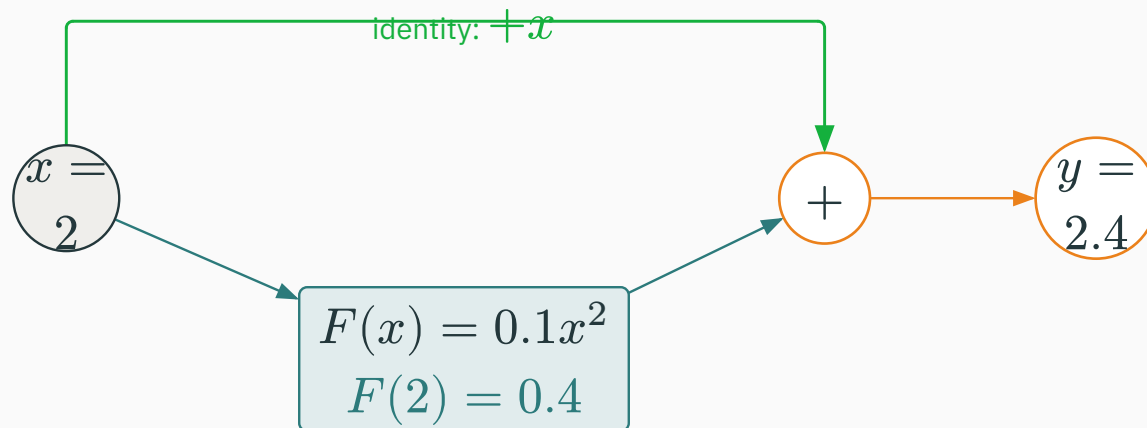
If “do nothing” is best, the learned branch only needs $F(x) \approx 0$.

The shortcut makes identity behaviour explicit; the residual path learns only the needed change.

Commit: residual arithmetic on one scalar

QUESTION

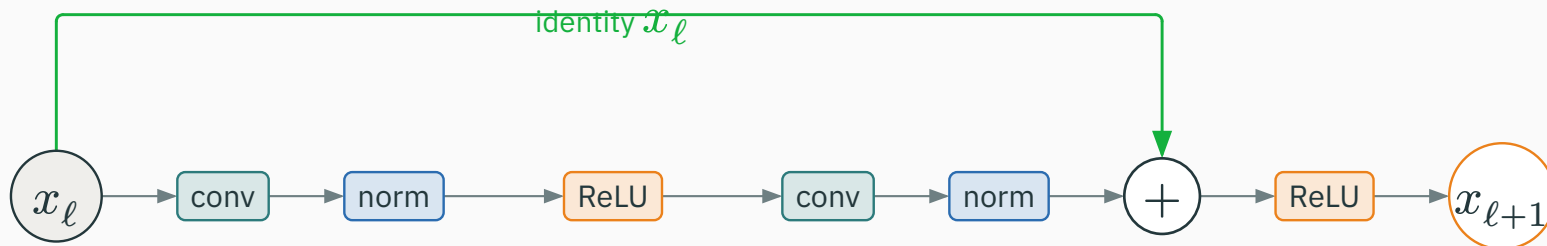
HELD CASE · ONE SCALAR BLOCK At $x = 2$, let $F(x) = 0.1x^2$. Predict $F(2)$, $y = x + F(x)$, and $d\frac{y}{d}x$.



$$F(2) = 0.1(2)^2 = 0.4, \quad y = x + F(x) = 2 + 0.4 = 2.4.$$

The output keeps the original 2 and adds the learned correction 0.4.

The residual block as a node diagram



$F(x_\ell)$ travels through learned layers; x_ℓ travels unchanged along the green edge.

The addition node is only legal when the two incoming tensors have identical shapes.

Why the gradient has a direct route

For $y = x + F(x)$,

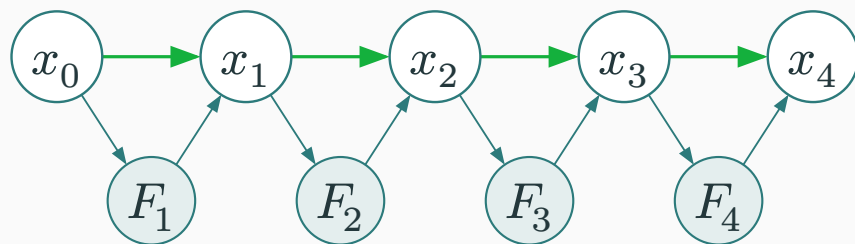
$$\frac{\partial y}{\partial x} = 1 + \frac{\partial F}{\partial x}.$$

Continue the scalar example $F(x) = 0.1x^2$ at $x = 2$:

$F'(x) = 0.2x$, so $F'(2) = 0.4$ and

$$\frac{dy}{dx} = 1 + 0.4 = 1.4.$$

The 1 is a direct derivative term. It helps signal flow; it does not guarantee a large total gradient if other terms cancel.



$$x_4 = x_0 + F_1(x_0) + F_2(x_1) + F_3(x_2) + F_4(x_3)$$

Depth becomes a sequence of refinements around a persistent representation.

The held channel change makes an identity shortcut illegal

QUESTION

HELD CASE · CANDIDATE 4 $x \in \mathbb{R}^{28 \times 28 \times 64}$ but the required stage output is $28 \times 28 \times 128$. Can $x + F(x)$ be added directly?

$$F(x) : 28 \times 28 \times 64 \rightarrow 28 \times 28 \times 128.$$

The identity tensor cannot be added as-is. Apply a 1×1 , stride-1 projection:

$$P(x) : 28 \times 28 \times 64 \rightarrow 28 \times 28 \times 128.$$

Now the addition is well-defined:

$$y = F(x) + P(x).$$

Projection shortcuts change shape; identity shortcuts preserve it.

A projected residual candidate still clears the budget

D DERIVATION

Use $F : 64 \rightarrow 8 \rightarrow 8 \rightarrow 128$ with $1 \times 1, 3 \times 3, 1 \times 1$; use $P : 64 \rightarrow 128$ with 1×1 . Biases are omitted.

$$\text{weights}_F = 64 \cdot 8 + 9 \cdot 8 \cdot 8 + 8 \cdot 128 = 2,112,$$

$$\text{weights}_P = 64 \cdot 128 = 8,192.$$

candidate	output	RF	weights	MACs
projected residual	$28 \times 28 \times 128$	3×3	10,304	8,078,336

$y = F(x) + P(x)$ is legal; both routes end at the same shape.

The shortcut is not free when shape changes—but this narrow residual branch remains within budget.

Misconception check: is the shortcut “skipping learning”?

A ANSWER

No. The output still depends on the learned residual $F(x)$.

$$x + F(x) = y$$

preserved representation + learned update

The shortcut changes the **parameterization** of the target function. It makes preservation cheap while leaving the branch free to learn large or small changes.

Residual learning is controlled refinement—not a bypass around the model.

Core move · factor spatial filtering from channel mixing

A standard $k \times k$ convolution does both jobs at once:

$$k^2 \cdot C_{\text{in}} C_{\text{out}}$$

$$k^2 \text{ spatial positions} \quad \times \quad C_{\text{in}} C_{\text{out}} \text{ channel pairs}$$

Ignoring bias, its weight count is $k^2 C_{\text{in}} C_{\text{out}}$; its MAC count is

$$H_{\text{out}} W_{\text{out}} k^2 C_{\text{in}} C_{\text{out}}.$$

Depthwise separable convolution performs the two jobs in two explicit steps.

Standard convolution: establish the baseline

Let $H = W = 28$, $k = 3$, $C_{\text{in}} = 64$, and $C_{\text{out}} = 128$.

$$\text{weights} = 3 \cdot 3 \cdot 64 \cdot 128 = 73,728.$$

There are $28 \cdot 28 = 784$ output locations:

$$\text{MACs} = 784 \cdot 73,728 = 57,802,752.$$

This is the cost the factorized version must match in output shape—not in arithmetic.

Depthwise step: one spatial filter per input channel

D DERIVATION

$$28 \times 28 \times 64 \xrightarrow{64 \text{ separate } 3 \times 3 \text{ filters}} 28 \times 28 \times 64$$

$$\text{weights} = 3 \cdot 3 \cdot 64 = 576,$$

$$\text{MACs} = 784 \cdot 576 = 451,584.$$

No channel is mixed with another yet.

Depthwise convolution answers only: “what spatial pattern occurs within this channel?”

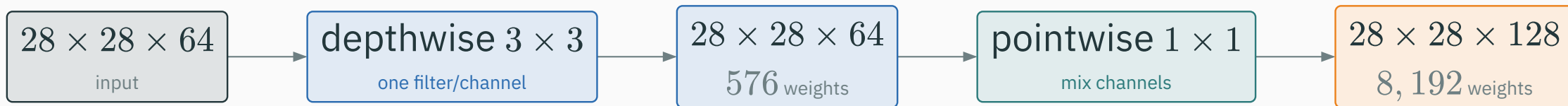
$$28 \times 28 \times 64 \xrightarrow{1 \times 1, 128 \text{ outputs}} 28 \times 28 \times 128$$

$$\text{weights} = 64 \cdot 128 = 8,192,$$

$$\text{MACs} = 784 \cdot 8,192 = 6,422,528.$$

The pointwise layer answers: “which channel combinations should form the 128 outputs?”

Put the factorized block back together



$$\text{separable weights} = 576 + 8,192 = 8,768,$$

$$\text{separable MACs} = 451,584 + 6,422,528 = 6,874,112.$$

$$\frac{57,802,752}{6,874,112} \approx 8.41 \times \text{fewer MACs than the standard convolution.}$$

one group $\rightarrow$ a standard convolution mixes **all** input channels

g groups $\rightarrow$ each output sees only $\frac{C_{\text{in}}}{g}$ input channels

$g = C_{\text{in}} \rightarrow$ depthwise connectivity: each output sees one input channel

With depth multiplier 1, $C_{\text{out}} = C_{\text{in}}$ and there is one spatial filter per channel; a later 1×1 restores full mixing.

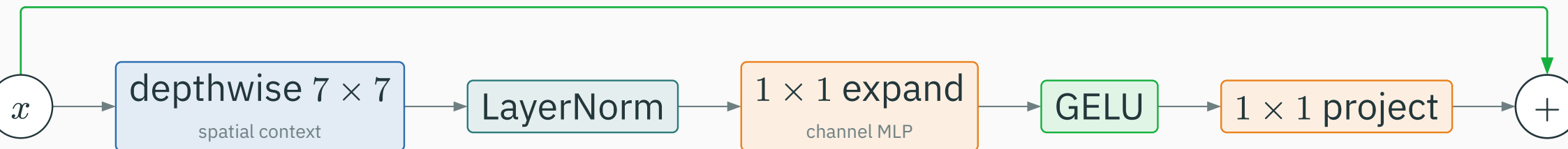
For a representative convolution, MACs scale approximately as

$$\text{MACs} \propto r^2 d w^2,$$

where r scales resolution, d depth, and w channel width.

Double	What changes	rough MAC multiplier
resolution r	$H, W \rightarrow 2H, 2W$	$4 \times$
depth d	twice as many blocks	$2 \times$
width w	$C_{\text{in}}, C_{\text{out}} \rightarrow 2 \times$	$4 \times$

EfficientNet's compound scaling grows all three gradually instead of overspending on one axis.



Modern CNNs recombine the same ideas: large-context depthwise filtering, cheap channel mixing, normalization, and a residual path.

Does $8.41 \times$ fewer MACs guarantee $8.41 \times$ lower wall-clock latency?

Not necessarily.

Wall-clock speed also depends on memory traffic, kernel launch overhead, tensor layout, batch size, and hardware support.

algorithmic cost $\neq$ measured latency

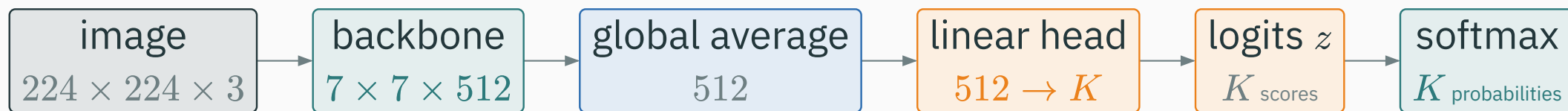
Use MACs and parameters for reasoning; benchmark latency on the deployment device before making the final choice.

The stage ledger now contains architectural choices

candidate	output	RF	weights	MACs
standard 3×3	$28^2 \times 128$	3×3	73,728	57.803M
two 3×3 s	$28^2 \times 128$	5×5	110,592	86.704M
bottleneck 16	$28^2 \times 128$	3×3	5,376	4.215M
Inception	$28^2 \times 128$	up to 5×5	9,920	7.777M
projected residual	$28^2 \times 128$	3×3	10,304	8.078M
depthwise + pointwise	$28^2 \times 128$	3×3	8,768	6.874M

The best block is conditional: satisfy the task contract, then measure the resource that matters on the target hardware.

A stage becomes one repeated part of a backbone



STAGES

POOL

HEAD

spatial transforms build a hierarchy

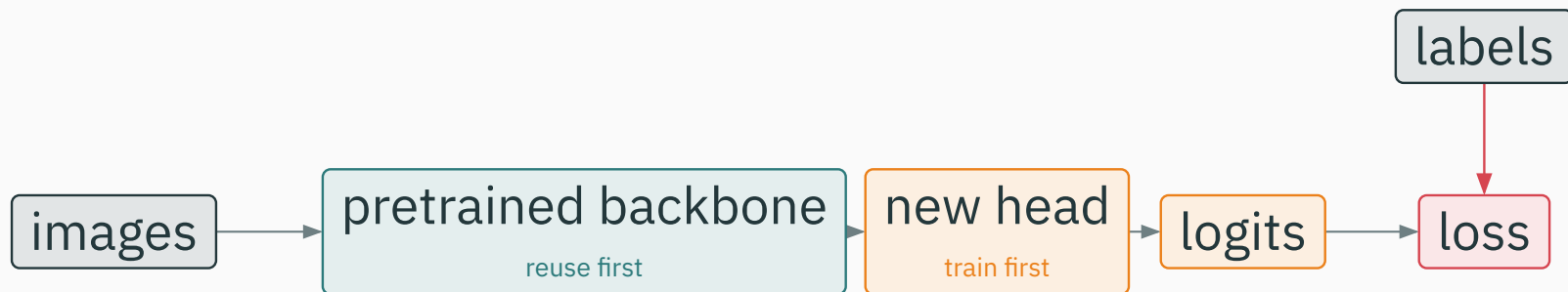
$$7 \times 7 \times 512 \rightarrow h \in \mathbb{R}^{512}$$

$$h \rightarrow z \in \mathbb{R}^K$$

The block ledger governs the backbone; transfer asks how much of that learned hierarchy to reuse.

Transfer starts with a hypothesis: old features may already separate new labels

pixels → edges and colours → textures → parts → task evidence



Probe the reused representation first; adapt it only when validation evidence says the fixed features are insufficient.

Commit: probe, unfreeze late, or fine-tune all?

QUESTION

A pretrained backbone emits $h \in \mathbb{R}^{512}$ for a new six-class dataset. Training data are limited; the test set is sealed.

A · PROBE

freeze the backbone;
train only $512 \rightarrow 6$

B · LATE BLOCKS

train the head and final
backbone stage

C · ALL

fine-tune every
parameter immediately

Commit before seeing target validation curves. The correct choice is empirical, not a slogan.

Count the new six-class head exactly

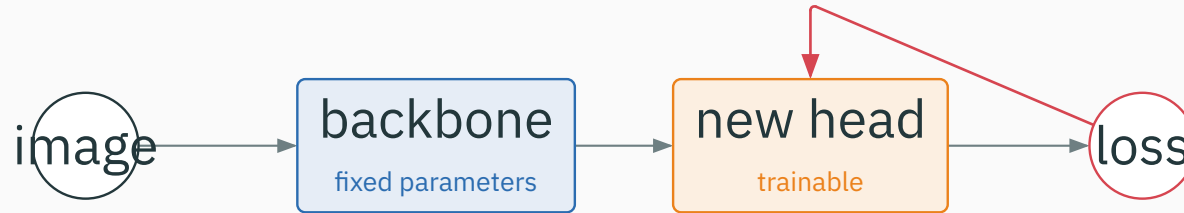
D DERIVATION

$$z = Wh + b, \quad W \in \mathbb{R}^{6 \times 512}, \quad b \in \mathbb{R}^6.$$

$$\text{trainable head parameters} = 6 \cdot 512 + 6 = 3,078.$$

If the backbone has 11 million parameters, a linear probe trains $\frac{3,078}{11,000,000 + 3,078} \approx 0.028\%$ of the model.

The first transfer experiment is intentionally tiny and inspectable.



A probe measures linear separability in pretrained feature space; it does not make backbone computation disappear.

SHOULD · freezing parameters does not freeze model state

D DERIVATION

PARAMETERS

`requires_grad=False`;
exclude them from the
optimizer.

BUFFERS

BatchNorm running
means/variances can still
change in `train()` mode.

STOCHASTIC LAYERS

Dropout also changes
behaviour between
`train()` and `eval()`.

For a truly fixed probe, keep the frozen backbone in `eval()` while the new head trains. Updating BatchNorm statistics deliberately is a different intervention—name and validate it.

SHOULD · match both the preprocessing and tensor-layout contracts

D DERIVATION

BOARD NOTATION

$H \times W \times C$ keeps the spatial story readable.

PYTORCH INPUT

Conv2d receives $N \times C \times H \times W$.

$$x'_c = \frac{x_c - \mu_c}{\sigma_c}$$

Channel order, input range, resize/crop policy, and (μ, σ) must match the weight recipe.

Before blaming transfer, inspect one batch: shape, dtype, range, channel order, normalization, and backbone mode.

TRAIN

fit head/backbone
parameters for one
declared regime

VALIDATION

choose probe vs late/all,
learning rates, stopping
epoch, and checkpoint

SEALED TEST

run once after the full
decision rule and
checkpoint are frozen

**Repeatedly checking test performance while choosing unfreeze depth
turns the test set into validation data.**

A controlled transfer experiment changes one lever at a time

Hold fixed the seed, train/validation split, head initialization, augmentation, optimizer family, epoch budget, and selection metric.

regime	trainable blocks	backbone LR	record
probe	head only	0	best validation; epoch
late	head + final stage	small	same record
all	entire network	smaller still	same record

Validation compares interventions fairly; the sealed test evaluates only the selected procedure.

Unfreeze progressively in response to evidence

probe underfits train and validation → reused features may not separate the task

unfreeze late blocks → test whether task-specific adaptation closes both gaps

consider broader unfreezing → only as another validation-controlled candidate

High training accuracy but poor validation is not evidence to unfreeze more. It points first to data size, regularization, augmentation, or shift.

Parameter groups make the fine-tuning hypothesis explicit

D DERIVATION

$$\theta_h \xrightarrow{\eta_h} \theta_h - \eta_h \nabla_{\theta_h} \mathcal{L}, \quad \theta_b \xrightarrow{\eta_b} \theta_b - \eta_b \nabla_{\theta_b} \mathcal{L}$$

$$0 < \eta_b < \eta_h$$

NEW HEAD

REUSED BACKBONE

larger step: it starts random and must learn the task

smaller step: preserve useful features while adapting

The ratio is a hyperparameter selected on validation—not a universal constant.

symptom	suspect	smallest discriminating test
loss stays near random	input contract mismatch	log batch shape/range; compare expected transform
frozen features drift	BN buffers or dropout active	diff parameters and buffers across one step
validation drops after unfreeze	backbone rate too large	reduce η_b ; log layer-wise update norms
MACs fall; latency does not	memory/kernel overhead	benchmark the target device and batch size

Change the experiment only after the test separates competing causes.

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L08B/01_depthwise_separable_and_transfer_head.ipynb

Open the exact Colab notebook. Predict the 73,728 / 8,768 parameter ratio, verify standard and separable NCHW paths, inspect residual identity gradients, then prove which parameters changed after one optimizer step.

FACTOR

57.803M $\rightarrow$ 6.874M MACs

PRESERVE

$x + F(x)$ and the direct gradient term

REUSE

512 $\rightarrow$ 6 head: 3,078 trainable

The synthetic TinyBackbone verifies freezing mechanics; because it is randomly initialized, it is not evidence that pretrained transfer improves validation.

The transfer decision is small, but the protocol is complete

START

512 → 6 head = 3,078 trainable
parameters

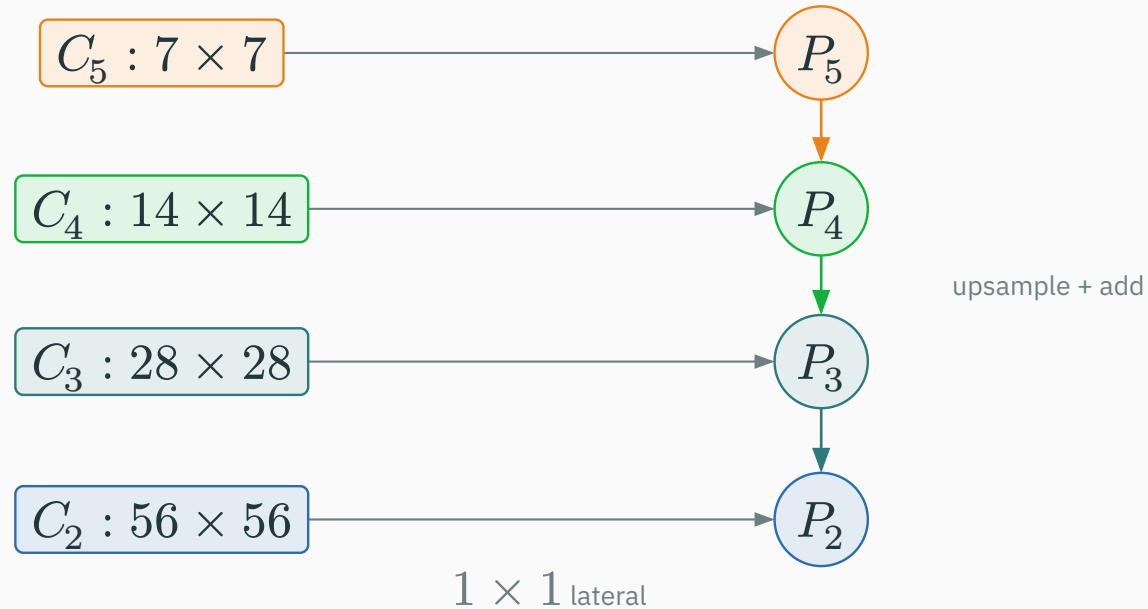
SELECT

validation chooses frozen/late/all and
the checkpoint

REPORT

sealed test once; include trainable
count and measured latency

“Transfer learning worked” means a controlled candidate won on validation and survived the one-time test—not merely that training ran.



A feature pyramid laterally projects backbone stages, upsamples coarse features, and adds only tensors with matching height, width, and channels.

Today the backbone is the reusable object. L10 will derive the localization heads and pyramid operations needed for detection.

need more context → stack small kernels (**VGG**)

need cheaper channel work → squeeze with 1×1 (**bottleneck**)

need several scales → branch then concatenate (**Inception**)

need trainable depth → add identity or projected shortcuts (**ResNet**)

need mobile efficiency → factor spatial and channel work (**MobileNet**)

need a new task → reuse backbone, replace head, validate adaptation

**Names provide provenance; shape, compute, route, and validation
provide the reasoning.**

Revisit the opening commitment before revealing it

HELD CASE · SAME DEPLOYMENT CONTRACT $28 \times 28 \times 64 \rightarrow 28 \times 28 \times 128$, at least 3×3 context, fewer than 10M convolution MACs.

A · STANDARD

73, 728 weights; 57.803M MACs

B · FACTORIZED

8, 768 weights; 6.874M MACs

C · EITHER

sharing alone guarantees budget

Defend your original choice using the ledger—not the architecture name.

$$\frac{57,802,752}{6,874,112} \approx 8.41 \times \text{fewer MACs}$$

SHAPE

$28 \times 28 \times 128$ on both paths

CONTEXT

depthwise 3×3 supplies the
required RF

BUDGET

$6.874\text{M} < 10\text{M}$

Factorization won this contract. A bottleneck, Inception, or projected residual candidate may win when context, branching, or optimization is the dominant need.

Given an image task, ask in order:

1. What output shape and receptive field must the evidence satisfy?
2. Which operation mixes space; which mixes channels?
3. Are additions and concatenations shape-legal?
4. Which weights and output positions dominate MACs?
5. Does measured device latency agree with the ledger?
6. Can a pretrained backbone be probed before broader adaptation?
7. Which validation rule chooses the regime before the test is opened?

A strong CNN pipeline is a sequence of declared contracts and measured decisions—not a pile of named blocks.

CLASSIFICATION

$7 \times 7 \times 512 \xrightarrow{\text{global average}} 512 \rightarrow 6$
one image-level answer

→

DETECTION

$C_2, C_3, C_4, C_5 \rightarrow$ spatial heads
classes **and** boxes at many locations

The next lecture keeps the reusable backbone but changes the head, targets, geometry, and evaluation.

BLOCKS

Simonyan & Zisserman · VGG
Szegedy et al. · Inception
He et al. · ResNet
Howard et al. · MobileNet

SCALING + PIPELINES

Tan & Le · EfficientNet
Liu et al. · ConvNeXt
Lin et al. · FPN
Exact L08B Colab notebook

Use the papers for historical claims; use the held ledger and notebook for reproducible arithmetic.